

رودمپ یادگیری پایتون تا شی‌گرایی

یک نقشه قدم‌به‌قدم برای کسی که می‌خواهد پایتون را درست یاد بگیرد؛ نه فقط سینتکس، بلکه تفکر برنامه‌نویسی، تمرین هدفمند و آمادگی برای ورود به پروژه‌های واقعی.

زمان روزانه
۶۰ تا ۹۰ دقیقه

نقطه پایان
کلاس و پروژه OOP

سطح شروع
مطلق مبتدی

مدت پیشنهادی
۱۰ تا ۱۲ هفته

هدف این مسیر این است که بتوانی یک مسئله روزمره را خرد کنی، با توابع و کلاس‌ها مدل کنی، و از صفر یک پروژه کوچک و قابل توضیح بسازی.

مسیر هشت مرحله‌ای

3 ساختمان داده

2 کنترل جریان

1 مبانی زبان

0 نصب و محیط

7 شی‌گرایی

6 پایتون میانی

5 فایل و خطا

4 توابع

یکی یکی

ده دوره همزمان نبین؛ یک مسیر را تمام کن.

خطا را بخوان

معلم توست؛ از آن Traceback فرار نکن.

پروژه محور

تا تمرین همان فاز را نساختی سراغ فاز بعد نرو.

هر روز کد

حتی بیست دقیقه بهتر از یک جلسه طولانی در هفته است.

چطور از این رودمپ استفاده کنی

این فایل یک دوره ویدیویی نیست؛ یک نقشه است. اگر هر هفته را تمام نشده رها کنی و به سراغ موضوع بعدی بروی، فقط توهم پیشرفت می‌گیری. قانون کار این است: تا پروژه همان فاز را نساختی، فاز بعد را شروع نکن.

روش پیشنهادی هر جلسه

- ۱۰ دقیقه مرور دیروز
- ۲۵ دقیقه یادگیری مفهوم جدید
- ۳۰ دقیقه تمرین دستی بدون کپی
- ۱۰ دقیقه بازنویسی تمیزتر همان کد
- ۵ دقیقه نوشتن خلاصه در دفتر یا فایل

قانون‌های طلایی

- هر روز کد بنویس؛ حتی ۲۰ دقیقه بهتر از صفر است.
- ویدیو را فقط برای فهمیدن ببین، نه برای جلو زدن.
- هر مفهوم را با یک مثال شخصی تمرین کن.
- خطا دیدن بخشی از یادگیری است، نه شکست.
- یادداشت کوتاه روزانه: چه یاد گرفتم، کجا گیر کردم.

معیار تسلط هر فاز: بتوانی موضوع را برای یک دوست توضیح بدهی، بدون نگاه به جزوه یک تمرین متوسط را حل کنی، و پروژه همان فاز را از صفر بنویسی.

فهرست مسیر

فاز	عنوان	زمان	خروجی یادگیری
فاز ۰	راه‌اندازی محیط کار	۳ تا ۵ روز	نصب، ترمینال، اولین اجرا
فاز ۱	مبانی زبان	۱/۵ هفته	متغیر، نوع داده، رشته
فاز ۲	عملگر و کنترل جریان	۱/۵ هفته	شرط، حلقه، منطق
فاز ۳	ساختمان‌های داده	۲ هفته	لیست، تاپل، ست، دیکشنری
فاز ۴	توابع	۱/۵ هفته	پارامتر، بازگشت، دامنه
فاز ۵	فایل، خطا، ماژول	۱/۵ هفته	کار واقعی با برنامه
فاز ۶	پایتون میانی	۱/۵ هفته	و ابزارها comprehension
فاز ۷	شی‌گرایی	۲/۵ هفته	کلاس تا چندریختی و پروژه

نمای کلی ۱۲ هفته

اگر روزی ۶۰ تا ۹۰ دقیقه وقت بگذاری، این جدول یک سرعت سالم است. اگر شاغل هستی، همان محتوا را در ۱۶ هفته پخش کن؛ عجله ارزش ندارد.

هفته ۱ نصب + مبانی	هفته ۲ رشته و ورودی	هفته ۳-۴ شرط و حلقه	هفته ۵-۶ ساختمان داده	هفته ۷ توابع	هفته ۸ فایل و خطا	هفته ۹ مبانی	۱۰-۱۲ پروژه + OOP
-----------------------	------------------------	------------------------	--------------------------	-----------------	----------------------	-----------------	----------------------

هفته	تمرکز	خروجی قابل لمس	نشانه آمادگی
۱	نصب پایتون، ویرایشگر، چاپ و متغیر	اسکرپت سلام + ماشین حساب ساده	بدون ترس برنامه را اجرا می‌کنی
۲	انواع داده و رشته	قالب‌بندی متن و تبدیل نوع	فرق عدد ۱۲ با رشته ۱۲ را می‌فهمی
۳	شرط و عملگر منطقی	برنامه تصمیم‌گیر: نمره، ورود، تخفیف	شاخه‌های تو در تو را می‌خوانی
۴	حلقه‌ها	جدول ضرب، حدس عدد، شمارش	می‌دانی حلقه کی تمام می‌شود
۵	لیست و تاپل	لیست خرید و نمرات کلاس	ایندکس و برش را بلدی
۶	دیکشنری و مجموعه	دفترچه مخاطب ساده	جستجو با کلید برایت طبیعی است
۷	توابع	کد تکراری را به تابع تبدیل کرده‌ای	ورودی و خروجی تابع را جدا می‌فهمی
۸	فایل، استثنا، ماژول	ذخیره و خواندن داده از فایل	برنامه با ورودی بد نمی‌ترسد
۹	ابزارهای میانی	کد کوتاه‌تر و خواناتر	را آگاهانه comprehension استفاده می‌کنی
۱۰	کلاس و شیء	مدل کردن یک موجودیت واقعی	معنی self برایت گنگ نیست
۱۱	ارث‌بری و کپسوله‌سازی	سلسله‌مراتب کلاس	تکرار کد بین کلاس‌ها کم شده
۱۲	پروژه شی‌گرا	یک برنامه چندفایلی کامل	می‌توانی معماریات را توضیح بدهی

اگر گیر کردی موضوع را ریزتر کن. به جای «لیست را بلد نیستم» بنویس «فرق append و extend را نمی‌فهمم». سؤال دقیق، یادگیری را نصف می‌کند.

فاز ۰ – راه‌اندازی محیط کار

۳ تا ۵ روز · هدف: بتوانی یک فایل پایتون بنویسی، ذخیره کنی و اجرا کنی؛ بدون وابستگی به سایت‌های آنلاین.

0

نصب، ویرایشگر و اولین اجرا

پایتون ۳/۱۱ یا جدیدتر · VS Code یا PyCharm · ترمینال

۳-۵ روز

تمرین روزانه

- یک پوشه به نام python-start بساز
- فایل hello.py را بنویس و اجرا کن
- عمداً یک غلط املایی بگذار و پیام خطا را بخوان
- همان برنامه را از ترمینال اجرا کن

یاد بگیر

- تفاوت مفسر و ویرایشگر
- نصب پایتون و بررسی نسخه
- ساخت پوشه پروژه
- اجرای فایل py و کار خیلی ساده با REPL

مینی‌پروژه: برنامه‌ای که اسمت را چاپ کند و یک خط توضیح بدهد امروز چرا پایتون یاد می‌گیری.

اشتباه رایج: یادگیری را داخل مفسر آنلاین شروع کردن و هیچ‌وقت محیط محلی نساختن. بعداً برای فایل، پکیج و پروژه گیر می‌کنی. از روز اول روی سیستم خودت کار کن.

فاز ۱ – مبانی زبان

حدود ۱۰ روز · هدف: داده را بگیری، نگه داری، تبدیل کنی و به‌شکل خوانا نشان بدهی.

1

متغیر، نوع داده، ورودی و رشته

int · float · str · bool · type conversion · f-string

۵/۱ هفته

تمرین‌های اجباری

- برنامه سن از سال تولد
- تبدیل دما از سانتی‌گراد به فارنهایت
- ساخت ایمیل فرضی از نام و نام خانوادگی
- شمارش کاراکتر یک جمله بدون فاصله
- گرد کردن قیمت به دو رقم اعشار

سرفصل‌ها

- و چند آرگومان print
- نام‌گذاری متغیر: خوانا و معنادار
- انواع اصلی و تابع type
- همیشه رشته برمی‌گرداند input
- تبدیل int / float / str
- رشته: ایندکس، برش، متدها
- strip, replace, split, join
- و قالب‌بندی عدد f-string
- مقادیر حقیقی و غلط، مقدار None

پروژه فاز: کارت معرفی شخصی. نام، سن، شهر و یک جمله هدف را بگیرد و در چند خط مرتب چاپ کند.

```
name = input("Your name: ").strip()
year = int(input("Birth year: "))
age = 1404 - year
print(f"Hello {name}! You are about {age}.")
```

فاز ۲ – عملگرها و کنترل جریان

حدود ۱۰ روز · هدف: برنامه تصمیم بگیرد و کار را تکرار کند. اینجا تفکر الگوریتمی شروع می‌شود.

2

شرط، منطق و حلقه

if / elif / else · and or not · for · while · break / continue

۱/۵ هفته

الگوی فکر کردن

- ورودی چیست؟
- چه حالت‌هایی وجود دارد؟
- هر حالت چه خروجی دارد؟
- تکرار تا کی باید ادامه پیدا کند؟
- حالت لبه‌ای چیست؟ صفر، منفی، رشته خالی

سرفصل‌ها

- عملگرهای حسابی و اولویت
- مقایسه و منطق and / or / not
- شاخه‌بندی چندگانه و شرط تو در تو
- range روی رشته و for
- و شرط توقف while
- break و continue
- حلقه تو در تو برای جدول و الگو
- اختیاری: match-case

پروژه فاز: خودپرداز خیلی ساده. موجودی اولیه داشته باشد، برداشت / واریز / نمایش موجودی را در حلقه انجام دهد و با انتخاب خروج تمام شود.

```
secret = 7
for attempt in range(1, 4):
    guess = int(input("Guess 1-10: "))
    if guess == secret:
        print("You won!")
        break
    print("Wrong.")
else:
    print("No more tries.")
```

نکته حلقه: قبل از نوشتن while روی کاغذ بنویس شرط خروج چیست. بیشتر حلقه‌های بی‌نهایت به‌خاطر فراموش کردن به‌روزرسانی متغیر کنترل است.

فاز ۳ – ساختمان‌های داده

۲ هفته · هدف: مجموعه‌ای از داده را نگه داری، جستجو کنی، تغییر بدهی و بین ساختار درست انتخاب کنی.

3

لیست، تاپل، مجموعه، دیکشنری

این قلب برنامه‌های واقعی است؛ عجله نکن

۲ هفته

ساختار	چه زمانی	ویژگی کلیدی	باید بلد باشی
list	ترتیب مهم است و داده عوض می‌شود	قابل تغییر و ایندکس‌دار	append pop insert sort slicing
tuple	داده ثابت است؛ مختصات یا رکورد ساده	غیرقابل تغییر	و استفاده به‌عنوان کلید unpacking
set	یکتایی و عضویت سریع	بدون ترتیب تکراری	اتحاد، اشتراک، تفاضل
dict	جستجو با کلید معنادار	جفت کلید-مقدار	get keys values items update

انتخاب ساختار

- اگر ترتیب مهم است: لیست
- اگر نباید عوض شود: تاپل
- اگر فقط عضویت مهم است: مجموعه
- اگر با نام یا شناسه پیدا می‌کنی: دیکشنری

تمرین‌ها

- حذف تکراری‌های یک لیست
- میانگین، بیشترین و کمترین نمره
- ادغام دو دیکشنری
- شمارش تکرار کلمات یک جمله
- لیست تو در تو: جدول دانش‌آموزان

پروژه فاز: مدیر کارهای روزانه. افزودن، حذف، علامت‌زدن انجام‌شده، نمایش باقی‌مانده‌ها. داده در لیست دیکشنری باشد؛ مثلا title و done.

فاز ۴ – توابع

حدود ۱۰ روز · هدف: کد را به قطعه‌های قابل استفاده مجدد بشکنی. این پیش‌نیاز مستقیم شی‌گرایی است.

4

تعریف، پارامتر، بازگشت و دامنه

def · return · default · *args · **kwargs · scope

۱/۵ هفته

سرفصل‌ها

- چرا تابع می‌نویسیم
- تفاوت چاپ کردن و برگرداندن
- پارامتر موقعیتی و کلیدواژه‌ای
- مقدار پیش‌فرض
- args و kwargs
- دامنه محلی و سراسری
- یک خطی docstring
- توابع کمکی کوچک بهتر از یک تابع غول‌آسا

قانون طراحی

- یک تابع، یک مسئولیت
- نام فعل‌دار مثل calculate_tax
- ورودی مشخص، خروجی مشخص
- اثر جانبی را آگاهانه انتخاب کن
- تکرار دیدی، استخراج کن

```
def grade_of(score: int) -> str:
    """Convert a numeric score to a letter."""
    if score >= 90:
        return "A"
    if score >= 80:
        return "B"
    if score >= 70:
        return "C"
    return "F"
```

پروژه فاز: کتابخانه کوچک ابزار متن. توابع جدا برای شمارش واژه، برعکس کردن جمله، حذف فاصله اضافه، و تشخیص پالیندروم. یک منو آن‌ها را صدا بزنند.

فاز ۵ – فایل، خطا و مازول

حدود ۱۰ روز · هدف: برنامه از حافظه فراتر برود، خطا را مدیریت کند و کد را در چند فایل سازمان بدهد.

۱/۵ هفته

ورود به برنامه‌های پایدار 5

open · with · json · try/except · import · venv · pip

تمرین

- خواندن یک فایل و شمارش خطوط
- ثبت گزارش روزانه در فایل
- برنامه با ورودی غلط پیام فارسی بدهد
- تقسیم توابع فاز ۴ بین دو فایل
- ذخیره لیست کارها در JSON

سرفصل‌ها

- خواندن و نوشتن فایل متنی
- چرا with از close بهتر است
- مسیر نسبی و مطلق
- ذخیره ساخت‌یافته با json
- try / except / else / finally
- گرفتن استثنای مشخص، نه همه چیز
- و ساخت مازول خودت import
- محیط مجازی و نصب بسته

```
from pathlib import Path
import json

path = Path("contacts.json")
try:
    contacts = json.loads(path.read_text(encoding="utf-8"))
except FileNotFoundError:
    contacts = []
```

پروژه فاز: دفترچه مخاطبین. افزودن، جستجو، حذف و ذخیره در contacts.json تا با بستن برنامه داده‌ها نپرد.

فاز ۶ – پایتون میانی

حدود ۱۰ روز · هدف: همان کارها را کوتاه‌تر، خواناتر و نزدیک‌تر به سبک حرفه‌ای بنویسی. این فاز را قبل از OOP حذف نکن.

۱/۵ هفته

ابزارهایی که کد را بزرگسال می‌کنند 6

comprehension · lambda · zip · enumerate · unpacking · type hints

قانون سلیقه

- خوانایی مهم‌تر از کوتاهی است
- تو در تو معمولاً بد است comprehension
- را برای منطق طولانی استفاده نکن lambda
- اگر بعد از یک ماه نفهمیدی‌اش، پیچیده بوده

سرفصل‌ها

- list / dict / set comprehension
- شرط داخل comprehension
- کی از comprehension استفاده نکنی
- enumerate و zip
- unpacking
- برای کلید مرتب‌سازی lambda
- sorted با key
- ساده برای توابع type hint
- آشنایی با any و all

```
scores = {"Ali": 18, "Sara": 16, "Nima": 19}
top = {name: s for name, s in scores.items() if s >= 18}
ranked = sorted(scores.items(), key=lambda item: item[1], reverse=True)
```

پروژه فاز: تحلیل‌گر نمرات کلاس. از یک دیکشنری یا فایل JSON میانگین، قبول‌شده‌ها، رتبه‌بندی و اختلاف از میانگین را با سبک میانی محاسبه کند.

قبل از ورود به شی‌گرایی این‌ها باید برای بدیهی باشند: متغیر و نوع داده، شرط و حلقه، لیست و دیکشنری، تابع با return، خواندن خطا، و شکستن مسئله به بخش‌های کوچک. اگر این‌ها لق می‌زنند، OOP را شروع نکن.

فاز ۷ – شی‌گرایی

۲/۵ هفته · هدف: دنیای واقعی را به کلاس و شیء مدل کنی و برنامه‌ای بسازی که گسترش آن سخت نباشد.

7

از تفکر تابعی به تفکر شیء

class · object · self · inheritance · encapsulation · polymorphism

۲/۵ هفته

شی‌گرایی یعنی داده و رفتار مربوط به آن را کنار هم بگذاری. به‌جای چند متغیر پراکنده برای حساب بانکی، یک کلاس حساب می‌سازی که موجودی را نگه می‌دارد و برداشت و واریز را بلد است.

هفته	موضوع	باید بتوانی
اول	کلاس، سازنده، ویژگی، متد، self	یک موجودیت واقعی را به کلاس تبدیل کنی
دوم	کپسوله‌سازی، property، class/static method	داده را محافظت و دسترسی را کنترل کنی
سوم	ارث‌بری، super، چندریختی، ترکیب	سلسله‌مراتب و جایگزینی شیء بسازی

ترتیب یادگیری دقیق

- نوشتن کلاس خالی و ساختن شیء
- سازنده و ذخیره ویژگی‌ها
- متدهایی که ویژگی را تغییر می‌دهند
- چاپ خوانا با متد مخصوص نمایش
- ویژگی خصوصی و property
- وراثت تک‌سطحی و super
- بازنویسی متد در زیرکلاس
- ترکیب به‌جای وراثت وقتی مناسب‌تر است
- چند کلاس در چند فایل

مفاهیم اصلی به زبان ساده

- کلاس: نقشه ساخت
- شیء: نمونه ساخته‌شده از آن نقشه
- ویژگی: داده‌ای که شیء دارد
- متد: کاری که شیء بلد است
- خود همان نمونه: self
- کپسوله‌سازی: جزئیات داخلی پنهان بماند
- ارث‌بری: کلاس جدید از کلاس قبلی
- چندریختی: یک پیام، رفتارهای متفاوت
- ترکیب: شیء از اشیای دیگر ساخته شود

```
class BankAccount:
    def __init__(self, owner: str, balance: int = 0):
        self.owner = owner
        self._balance = balance

    def deposit(self, amount: int) -> None:
        if amount <= 0:
            raise ValueError("amount must be positive")
        self._balance += amount

    def __str__(self) -> str:
        return f"{self.owner}: {self._balance}"
```

تمرین‌های پلکانی شی‌گرای

سطح ۱ – گرم‌کردن

- کلاس Book با عنوان، نویسنده، سال
- کلاس Student با افزودن نمره و میانگین
- کلاس Rectangle با مساحت و محیط
- کلاس Countdown که هر بار یک واحد کم شود

سطح ۲ – رفتار واقعی

- کلاس Playlist برای افزودن و پخش بعدی
- کلاس Cart فروشگاه با جمع کل
- کلاس User با اعتبارسنجی رمز
- جلوگیری از موجودی منفی در حساب بانکی

سطح ۳ – وراثت

- و زیرکلاس‌های ثابت‌حقوق و ساعتی Employee
- و ماشین / موتور با رفتار متفاوت Vehicle
- و محاسبه مساحت چندشکل Shape
- بازنویسی str در هر زیرکلاس

سطح ۴ – ترکیب چند کلاس

- کتابخانه: Book + Member + Library
- مدرسه: Student + Course + GradeBook
- فروشگاه: Product + Cart + Order
- هر کلاس در فایل جدا، یک فایل main.py

پروژه پایان مسیر (یکی را انتخاب کن): سیستم امانت کتابخانه • مدیریت حساب‌های بانکی چندنوعی • فروشگاه کوچک با سبد خرید و فاکتور • بازی متنی ساده با شخصیت، موجودی و مکان. شرط قبولی: حداقل ۳ کلاس مرتبط، ذخیره در فایل، منوی کاربری، مدیریت خطا، و توضیح شفاهی معماری در ۱۰ جمله.

اشتباه‌های کلاسیک در OOP

این کارها را نکن

- همه چیز را در یک کلاس خداگونه بریزی
- برای هر تابع یک کلاس بی‌دلیل بسازی
- وراثت عمیق چندسطحی راه بیندازی
- ویژگی‌ها را از بیرون مستقیم دستکاری کنی
- منطق برنامه را داخل print پنهان کنی

به‌جایش این کار را بکن

- هر کلاس یک مسئولیت روشن داشته باشد
- اول با توابع حل کن، بعد در صورت نیاز کلاس بساز
- ترکیب را به وراثت پیچیده ترجیح بده
- تغییر داده را از راه متد انجام بده
- متد مقدار برگرداند؛ نمایش را جدا کن

برنامه هفتگی پیشنهادی

این الگو را هر هفته تکرار کن. روز هفتم برای جلو زدن نیست؛ برای محکم کردن است.

روز	کار	زمان
شنبه	مفهوم جدید + مثال‌های کتاب یا دوره	۷۵ دقیقه
یکشنبه	همان مفهوم با تمرین‌های خودت	۷۵ دقیقه
دوشنبه	مفهوم بعدی + اتصال به دیروز	۷۵ دقیقه
سه‌شنبه	تمرین ترکیبی و بازنویسی کد قبلی	۷۵ دقیقه
چهارشنبه	شروع یا ادامه پروژه همان فاز	۹۰ دقیقه
پنجشنبه	تکمیل پروژه و رفع باگ	۹۰ دقیقه
جمعه	مرور، چک‌لیست، توضیح بلند برای خودت	۴۵ دقیقه

منابع پیشنهادی

چطور منبع انتخاب کنی

- یک دوره یا یک کتاب را تمام کن؛ ده منبع را همزمان نه
- منبعی خوب است که تمرین اجباری دارد
- اگر فقط نگاه می‌کنی و تایپ نمی‌کنی، منبع را عوض کن
- بعد از هر درس لپ‌تاپ را ببند و از حفظ همان را بنویس

رایگان و معتبر

- مستندات رسمی: docs.python.org/3/tutorial
- کتاب Automate the Boring Stuff برای انگیزه عملی
- تمرین: exercism.org/tracks/python
- پرسش و پاسخ: اول خودت خطا را بخوان
- برای OOP: بخش Classes در مستندات رسمی

تمرین‌ساز شخصی

هر مفهوم را با زندگی خودت گره بزن. اگر دانشجویی، نمره و واحد درس را مدل کن. اگر فروشگاه داری، کالا و فاکتور را مدل کن. یادگیری وقتی می‌چسبد که مسئله مال خودت باشد.

عادت حرفه‌ای از همین حالا: نام فایل و متغیر انگلیسی و معنادار، تورفتگی ۴ فاصله، و هر پروژه یک پوشه جدا با فایل README کوتاه.

چک لیست تسلط نهایی

اگر حدود ۸۰ درصد این موارد را بدون جزوه بلدی، آماده‌ای سراغ مرحله بعد بروی: کتابخانه‌های جانبی، API، یا پروژه بزرگ‌تر.

پایداری و OOP

- داده را در فایل ذخیره و بازیابی می‌کنم
- ورودی بد برنامه را خراب نمی‌کند
- کد را در چند ماژول تقسیم می‌کنم
- کلاس با سازنده و متد می‌نویسم
- فرق شیء و کلاس را توضیح می‌دهم
- ارث‌بری ساده و override بلدم
- یک پروژه چندکلاسی از صفر می‌سازم

مبانی تا توابع

- تفاوت نوع‌ها را بلدم و تبدیل می‌کنم
- شرط تو در تو را ساده می‌کنم
- حلقه را عمدا متوقف یا ادامه می‌دهم
- بین لیست و دیکشنری درست انتخاب می‌کنم
- تابع مقدار برمی‌گرداند، فقط چاپ نمی‌کند
- کد تکراری را استخراج می‌کنم

بعد از این رودمپ چه کار کنی

اگر عاشق اتوماسیون شدی

فایل، پوشه، اکسل و اسکریپت‌های روزانه. همین الان هم با فاز ۵ می‌توانی ابزارهای کوچک مفید بسازی.

اگر عاشق وب شدی

بعد از این مسیر، HTTP را بشناس و بعد Flask یا FastAPI. کلاس و ماژول اینجا مستقیما به کارت می‌آید.

اگر عاشق داده شدی

بعدا NumPy، pandas و مصورسازی. ولی تا را نساختی سراغ‌شان نرو؛ وگرنه فقط OOP دستور حفظ می‌کنی.

جمله آخر: برنامه‌نویس کسی نیست که سینتکس حفظ کرده باشد. کسی است که مسئله را خرد می‌کند، امتحان می‌کند، خطا را می‌خواند و دوباره می‌سازد. این رودمپ فقط مسیر است؛ حرکت با توست.

نقطه پایان این مسیر

وقتی بتوانی یک مسئله روزمره را به چند کلاس تقسیم کنی، در فایل ذخیره کنی و برای یک نفر دیگر توضیح بدهی چرا این‌طور ساختی، پایتون را تا شی‌گرایی بلدی.

پایان فایل - برای مطالعه چاپی و پیگیری هفتگی طراحی شده است.